

FOR INTERNSHIP CANDIDATES

Be Unstoppable.

Build small. Ship it. Tell us why.

A four-day DevOps internship case study — an end-to-end production slice, one deliverable per day.

Hoş geldin!

Welcome to the Insider One DevOps internship case study.

Bu vaka çalışması, üretim sistemlerine nasıl yaklaştığını anlamak için tasarlandı. Kusursuz bir sistem değil; net kararlar ve uçtan uca çalışan küçük bir parçayla ilgileniyoruz.

This case study is designed to understand how you approach production systems. The aim is clear decisions and a small, end-to-end working slice — not over-engineered perfection.

TIME BUDGET · SÜRE

Yaklaşık 4 iş günü. Her güne net bir teslimat düşünüyor. Hangi araçla nasıl yapacağına karar vermek sana kalmış; tıkanıp yerinde fikir almak istersen yazabilirsin.

About 4 working days, with one deliverable per day. You decide how to approach the work and which tools to use; feel free to reach out if you'd like a second opinion.

HOUSE RULE

AI asistanı kullanmakta tamamen özgürsün. Ne kullandığını ve karar gerekçelerini README ve karar kayıtlarında kısaca paylaşman yeterli.

Using AI assistants is totally fine. A brief note of what you used and your reasoning in the README and decisions log is plenty.

WHAT MATTERS · ÖNEMLİ OLAN

C

Clarity

Sade yapı, okunabilir kod, anlaşılır bir README.

Simple structure, readable code, a clear README.

A

Automation

Manuel adımlar yerine küçük de olsa çalışan bir CI/CD akışı.

A small but working CI/CD flow instead of manual steps.

A

Awareness

Sırlar, portlar, IAM, image taraması gibi konularda temel bir hassasiyet.

Basic care around secrets, ports, IAM, and image scanning.

D

Decisions

Kullandığın aracı neden seçtiğini birkaç cümleyle anlatmak.

A few sentences explaining each tool choice.

Takıldın mı? Soru sormak öğrenme sürecinin doğal bir parçası. Bir konuyu nasıl yapacağından emin değilsen, yaklaşımı birlikte konuşmaktan memnuniyet duyarız.

Stuck? Asking is a natural part of learning. If you're unsure how to approach something, we'd be happy to talk it through together.

Build a tiny app. Ship it like production.

Küçük bir servisi sıfırdan üretime alır gibi inşa etme süreci.

Küçük bir HTTP servisi yazıyorsun, Docker ile paketliyorsun, Helm chart ile minikube'a deploy ediyorsun, GitHub Actions ile CI/CD kuruyorsun, gözlemlenebilir hale getiriyorsun ve internete açıyorsun. Tek başına çalışan, tekrar üretilebilir bir uçtan uca parça.

You'll write a tiny HTTP service, package it with Docker, deploy it to minikube via Helm, wire up CI/CD with GitHub Actions, add observability, and expose it to the internet. A small, reproducible end-to-end slice.



HEADS UP · DIKKAT

Minikube tek-node bir Kubernetes ortamı — laptop üzerinde de, EC2 üzerinde de rahatlıkla koşar. Production seviyesinde bir kümeye ihtiyaç yok; Deployment, Service ve Ingress yeterli olur.

Minikube is a single-node Kubernetes environment — it runs comfortably on a laptop or on EC2. There's no need for a production-grade cluster; a Deployment, Service, and Ingress will be enough.

Cloud or local?

İki yoldan biri yeterli; ikisini birden yapmaya gerek yok.

TRACK A · CLOUD

Minikube on EC2

Running minikube on an AWS free-tier EC2 instance.

AWS free tier üzerinde minikube'a uygun bir EC2 instance kurarak uygulamayı deploy etme ve Elastic IP ile dışarıya açma yolu.

Setting up a free-tier EC2 instance that fits minikube, deploying the app on it, and exposing it through an Elastic IP.

- AWS free tier hesabı gerekiyor (aws.amazon.com/pm/ec2)
- Minikube'a uygun instance tipini bulmak sana kalmış — RAM ve CPU gereksinimini araştırmak iyi bir başlangıç
- Güvenlik grubu üzerinden NodePort veya 80/443 portlarının dışarıya açılması gerekecek
- Public URL şu formatta olacak: `http://<EIP>:<port>`
- IaC kullanmak istersen Terraform veya OpenTofu Track A kapsamında değerlendiriliyor

TRACK B · LOCAL

Minikube + tunnel

Running minikube locally, exposed through a public tunnel.

Kendi makinende minikube kurarak uygulamayı deploy etme ve NodePort veya Ingress'i ngrok ya da cloudflared aracılığıyla internete açma yolu.

Setting up minikube on your own machine, deploying the app, and exposing the NodePort or Ingress through ngrok or cloudflared.

- AWS hesabı gerekmiyor; bir laptop yeterli
- minikube ve kubectl ile Docker driver yeterli olacak
- Tunnel için ngrok'un ücretsiz planı veya cloudflared kullanılabilir
- Kurulum genellikle 15 dakika kadar sürüyor
- Day 4'teki IaC adımı yerine alternatif bir görev öneriliyor (ileride detayları var)

Hangisini seçmeli? · Which one to pick? AWS deneyimi kazanmak ilgini çekiyor ve free tier hesabın varsa Track A güzel bir seçim. Daha hızlı bir kurulum ve cloud maliyeti olmadan ilerlemek istiyorsan Track B uygun. Her ikisine de aynı şekilde bakıyoruz.

If you'd like some AWS exposure and have a free-tier account, Track A is a nice fit. If you'd rather move quickly with no cloud cost, Track B works just as well. Both are looked at the same way.

DAY 01 • FOUNDATION

App, container, repo

Yapı taşlarını yerine koyma günü: küçük bir servis, temiz bir Docker imajı, düzenli bir repo.

DELIVERABLE

Yerelde docker run ile çalışan bir servis ve GitHub repo

1.1 Tiny HTTP service

Node.js, Go veya Python ile küçük bir servis yazmak iyi bir başlangıç. Üç endpoint düşünüyoruz: **GET /ping** → pong, **GET /healthz** probe'lar için, **GET /version** build SHA bilgisi için.

A small service in Node.js, Go, or Python works well as a starting point. Three endpoints to think about: **GET /ping** returning pong, **GET /healthz** for probes, **GET /version** returning the build SHA.

core

node • go • python

env-driven config

1.2 Containerize

Multi-stage bir Dockerfile, root olmayan bir kullanıcı, küçük bir base image (distroless veya alpine güzel seçenekler) ve **HEALTHCHECK** ile küçük bir imaj çıkıyor. Yerel geliştirme için **docker-compose.yaml** hayatı kolaylaştırabilir.

A multi-stage Dockerfile, a non-root user, a small base image (distroless or alpine are nice options), and a **HEALTHCHECK** make for a tidy image. A **docker-compose.yaml** can make local development easier.

core

docker

multi-stage

non-root

1.3 Repo hygiene

Public bir GitHub reposu, içinde README.md, .gitignore ve .env.example olması iyi olur. main artı feature branch'leri ile çalışmak, conventional commits, PR template, CODEOWNERS ve branch protection ufak ama anlamlı dokunuşlar.

A public GitHub repo with README.md, .gitignore, and .env.example is a solid baseline. Working with main plus feature branches, conventional commits, a PR template, CODEOWNERS, and branch protection are small but meaningful touches.

core

readme

codeowners

pr-template

1.4 Minimal test

En az 1-2 unit test yeterli. Test framework seçimi tamamen sana kalmış. Bu testler Day 3'te CI'a bağlanacak, bugün yerelde çalışması yeterli.

One or two unit tests are enough. The choice of test framework is entirely up to you. These tests will be wired into CI on Day 3; for today, running them locally is enough.

core

unit test

Day-end checkpoint. Şu adımların çalıştığını görmek iyi bir bitiş: docker build, docker run, ardından curl ile /ping endpoint'inden pong yanıtı. git log temiz görünüyor olmalı; sırların repo'ya girmemiş olması da önemli.

A good way to wrap up: docker build, docker run, then a curl to the /ping endpoint returning pong. The git log looks clean and secrets are out of the repo.

DAY 02 · KUBERNETES & HELM

Deploy with Helm

Servisi paketleyip iki farklı ortama (dev ve prod) deploy etme günü; production'a yakın bir akış oluşuyor.

DELIVERABLE

helm upgrade --install ile çalışan deploy ve iki ortam arasında belirgin bir fark

2.1 Helm chart

Raw manifest yerine kendi Helm chart'ını oluşturmak güzel bir adım: Deployment, Service, Ingress, ConfigMap, Secret. helm create şablonundan başlayıp üzerine geliştirmek tamamen normal.

Building your own Helm chart instead of raw manifests is a nice step: Deployment, Service, Ingress, ConfigMap, Secret. Starting from helm create and building on it is perfectly fine.

core helm ingress

2.2 Environments

values-dev.yaml ve values-prod.yaml farkları belirgin olunca anlamlı oluyor; replica sayısı, kaynaklar, host gibi. README'de hangi değer hangi ortama bağlı olduğunu kısaca paylaşmak da işine yarayabilir.

values-dev.yaml and values-prod.yaml become meaningful when the differences are clear: replica count, resources, host, and so on. A short note in the README about which value goes with which environment is helpful too.

core values-dev values-prod

2.3 Probes & resources

liveness ve readiness probe'larını (isteğe bağlı olarak startup probe da) /healthz'a bağlamak güzel oluyor. Her container için requests ve limits belirlemek de iyi bir alışkanlık; değerleri nasıl seçtiğine dair kısa bir not README'de yeterli.

Pointing the liveness and readiness probes (optionally a startup probe too) at /healthz works nicely. Setting requests and limits per container is a good habit as well; a short note in the README on how the values were chosen is enough.

core probes limits

2.4 Rollout & rollback

İmaj tag'ini bilerek değiştirip helm upgrade ile yeni sürümü açmak, sonra helm rollback ile geri almak güzel bir egzersiz. Kanıt olarak helm history ve kubectl rollout status çıktıları iş görüyor.

Bumping an image tag, rolling out a new version with helm upgrade, and then rolling it back with helm rollback makes for a nice exercise. The output of helm history and kubectl rollout status works well as evidence.

core rollout rollback

2.5 Bonus (opsiyonel)

İstersen HPA (CPU bazlı, basit bir hedef), NetworkPolicy (sadece ingress-controller üzerinden trafik) veya PodDisruptionBudget eklemek güzel olur. Tercih ettiğin maddenin nedenini README'de kısaca paylaşmak yeterli.

If you'd like, adding an HPA (a simple CPU-based target), a NetworkPolicy (allowing traffic only from the ingress controller), or a PodDisruptionBudget is a nice touch. A short note in the README about why you picked it is plenty.

bonus hpa networkpolicy pdb

Day-end checkpoint. helm upgrade --install app -f values-dev.yaml komutundan sonra pod'lar Running ve probe'lar Healthy görünüyor. Aynı komut values-prod.yaml ile koşurulduğunda farklı replica ve host değerleri ortaya çıkıyor.

After helm upgrade --install app -f values-dev.yaml, the pods are Running and the probes are Healthy. The same command with values-prod.yaml produces different replica and host values.

DAY 03 · CI/CD & SUPPLY CHAIN SECURITY

Pipeline that protects you

Build, test, scan, push ve deploy adımlarını otomatize etme günü; ufak ama production seviyesinde güvenlik dokunuşlarıyla.

DELIVERABLE

Yeşil bir CI çalışması, GHCR üzerinde bir imaj ve etiketlenmiş bir v0.1.0 release

3.1 CI pipeline

GitHub Actions ile akış şöyle güzel oluyor: lint, test, docker build, Trivy ile imaj taraması (CRITICAL ve HIGH bulunursa pipeline kırılıyor) ve GHCR'a push. Workflow dosyaları .github/workflows/ altında, public repolarda free tier rahatlıkla yetiyor.

A nice GitHub Actions flow: lint, test, docker build, a Trivy image scan (failing on CRITICAL or HIGH), and a push to GHCR. Workflow files under .github/workflows/, free tier is plenty for public repos.

core

actions

trivy

ghcr

3.2 Secrets & auth

Track A için AWS'e OIDC federation üzerinden bağlanmak güzel bir yaklaşım; uzun ömürlü access key tutmaya gerek kalmıyor. gitleaks ile sırların taranması da iyi bir ek. Hedef: repo içinde gerçek bir credential bulunmaması.

For Track A, connecting to AWS via OIDC is a nice approach; no need for long-lived access keys. Adding gitleaks is a good extra. The goal: no real credentials inside the repo.

core

oidc

gitleaks

3.3 Release hygiene

Anlamli bir v0.1.0 tag, kısa bir CHANGELOG ve bir GitHub Release güzel bir set. İmaj tag'inin release ile aynı SHA ya da semver'i taşıması işleri tutarlı kılıyor.

A meaningful v0.1.0 tag, a short CHANGELOG, and a GitHub Release make a nice set. Keeping the image tag aligned with the release SHA or semver keeps things consistent.

core

semver

changelog

3.4 Auto-deploy on merge

main'e merge sonrası minikube'a otomatik deploy iki şekilde gidebilir: pipeline'dan kubectl set image ya da ArgoCD/Flux ile GitOps. Tercih ve tradeoff README'de güzel oluyor.

After merge to main, auto-deploying to minikube can go two ways: kubectl set image from the pipeline, or GitOps via ArgoCD/Flux. The choice and tradeoff fit nicely in the README.

core

argocd · flux

gitops

3.5 Bonus (opsiyonel)

İstersen cosign ile imaj imzalama, Syft ile SBOM üretimi, multi-arch build (amd64 + arm64) veya basit bir release-please otomasyonu eklemek güzel. Hepsisi opsiyonel.

If you'd like, adding cosign image signing, a Syft SBOM, a multi-arch build (amd64 + arm64), or a simple release-please automation is nice. All optional.

bonus

cosign

sbom

multi-arch

Day-end checkpoint. Bir PR açıldığında pipeline yeşil dönüyor, imaj GHCR'a iniyor, v0.1.0 release oluşuyor ve main'e merge sonrası minikube'da yeni imaj koşuyor.

A PR runs green, the image lands in GHCR, a v0.1.0 release is cut, and after merge to main the new image runs on minikube.

DAY 04 • OBSERVABILITY, IAC & DOCUMENTATION

Make it operable

Sistemi başka birinin de ayakta tutabileceği bir hale getirme günü: metrikler, alertler, runbook, mimari.

DELIVERABLE

Grafana dashboard ekran görüntüsü,
RUNBOOK, ADR ve bir mimari diyagram

4.1 Logs & metrics

Uygulamadan yapılandırılmış JSON loglar (timestamp, level, msg, request_id) güzel bir başlangıç. /metrics endpoint'i de Prometheus formatında birkaç varsayılan metrikle yetiyor.

Structured JSON logs from the app (timestamp, level, msg, request_id) make a nice start. A /metrics endpoint in Prometheus format with a few default counters is enough.

core json logs /metrics

4.2 Prometheus + Grafana

minikube üzerine kube-prometheus-stack Helm chart'ı ile kurulum güzel oluyor. En az bir Grafana dashboard (RPS, latency, hata oranı, pod restart gibi) ve en az bir alert kuralı (örneğin `rate(errors) > 5%`) yeterli.

Installing kube-prometheus-stack via Helm on minikube works nicely. At least one Grafana dashboard (RPS, latency, error rate, pod restarts) and at least one alert rule (for example `rate(errors) > 5%`) is enough.

core prometheus grafana alert

4.3 Infrastructure as Code

Track A için EC2, EIP ve güvenlik grubunu Terraform veya OpenTofu ile tanımlamak güzel (local state yeterli). Track B için minikube'un ayağa kalkışını ve gerekli komutları bir Makefile ya da basit bir shell script ile tekrar üretilebilir hale getirmek de aynı amaca hizmet ediyor.

For Track A, defining the EC2, EIP, and security group with Terraform or OpenTofu is nice (local state is fine). For Track B, making the minikube setup and the needed commands reproducible through a Makefile or a small shell script serves the same purpose.

core terraform • opentofu makefile

4.4 Architecture & docs

Excalidraw veya draw.io ile bir mimari diyagram (uygulama, container, Kubernetes, ingress, Public URL ve üzerine bir observability katmanı) güzel oluyor. Bir RUNBOOK.md (yeniden başlatma, logların nereden bakılacağı, geri alma adımları), bir SECURITY.md ve üç kadar ADR de tablonun geri kalanını tamamlıyor.

An architecture diagram in Excalidraw or draw.io (app, container, Kubernetes, ingress, the Public URL, with an observability overlay) works nicely. A RUNBOOK.md (restart steps, where to find logs, how to roll back), a SECURITY.md, and about three ADRs round out the rest.

core diagram runbook adr

Day-end checkpoint. Grafana üzerinde dashboard açılıyor ve en az bir alert tanımlı. RUNBOOK kısa bir olay müdahale rehberi gibi okunabiliyor (tek sayfa yeterli) ve ADR'lar "neden Helm", "neden bu base image", "neden bu tunnel" türünden soruları yanıtlıyor.

The Grafana dashboard opens and at least one alert is defined. The RUNBOOK reads like a short incident response guide (a single page is enough) and the ADRs answer questions like "why Helm", "why this base image", "why this tunnel".

+ OPTIONAL BONUS

Going further

Tamamen opsiyonel maddeler. Yapmamak değerlendirmeyi olumsuz etkilemiyor; yapıyorsan derinleşmek yerine küçük bir iz bırakmak yeterli.

Bu maddeler merak ettiğin bir konuyu keşfetmek için bir vesile. Ana akışı bitirdiysen bir tanesini seçip eklemek güzel; nasıl eklediğine ve ne öğrendiğine dair birkaç cümlelik bir not da README'de iş görüyor.

These items are a chance to explore something you're curious about. If you've finished the main flow, picking one and adding it is nice; a few sentences in the README about how you added it and what you learned is enough.

+ Policy-as-Code

Kyverno veya OPA Gatekeeper ile bir cluster policy: örneğin "no root containers" ya da "no :latest image tag".

A cluster policy via Kyverno or OPA Gatekeeper: for example, "no root containers" or "no :latest image tag".

+ Supply chain

CI'a cosign attestation ve Syft ile SBOM entegrasyonu; imaja SLSA tarzı meta veri ekleme.

Integrating cosign attestation and a Syft SBOM into CI; attaching SLSA-style metadata to the image.

+ GitOps proper

ArgoCD ApplicationSet ile dev ve prod ortamları, otomatik sync ve prune açık şekilde.

An ArgoCD ApplicationSet for dev and prod environments, with auto-sync and prune enabled.

+ Chaos test

Bir pod silindiğinde sistemin kendini iyileştirdiğini gözlemleme. kubectl-chaos veya basit bir loop da seçenek olabilir.

Observing the system self-heal when a pod is deleted. kubectl-chaos or a simple loop are options too.

+ Custom metric → alert → Slack

Uygulamaya alana özgü bir counter eklemek ve Prometheus, Alertmanager üzerinden Slack webhook'a uzanan bir akış kurmak.

Adding a domain-specific counter to the app and wiring a flow through Prometheus and Alertmanager to a Slack webhook.

+ Custom domain + TLS

Cloudflare veya Route53 üzerinden bir alan adı yönlendirmek ve cert-manager ile Let's Encrypt sertifikası almak.

Pointing a custom domain through Cloudflare or Route53 and obtaining a Let's Encrypt certificate via cert-manager.

Mükemmel olması gerekmiyor. Bir tanesini bile güzelce ele almak, hepsini yarım bırakmaktan çok daha değerli.

It doesn't need to be perfect. Doing one of them well is far more valuable than leaving them all half-baked.

What to send us

Teslim edeceğin parçalar — kısa ve net.

- ☐ **Public GitHub repo link**
Halka açık ya da bizi davet ettiğin private bir repo.
A public repo, or a private one you invite us to.
- ☐ **README.md**
Kurulum, çalıştırma, ortam değişkenleri, mimari notlar ve seçilen track.
Setup, run, env vars, architectural notes, and the chosen track.
- ☐ **Architecture diagram**
Excalidraw, draw.io ya da kağıt-kalem fotoğrafı, hangisi rahatsa.
Excalidraw, draw.io, or a photo of a paper sketch — whichever feels comfortable.
- ☐ **Helm chart klasörü**
values-dev.yaml ve values-prod.yaml dahil olacak şekilde.
Including values-dev.yaml and values-prod.yaml.
- ☐ **CI/CD workflow dosyaları**
.github/workflows/ altında yeşil bir koşunun kanıtıyla birlikte.
Under .github/workflows/, with evidence of a green run.
- ☐ **kubectİ ve helm çıktıları**
get pods, helm list, helm history ve rollout status ekran görüntüleri.
Screenshots of get pods, helm list, helm history, and rollout status.
- ☐ **Grafana dashboard ekran görüntüsü**
En az bir dashboard ve bir alert kuralı görünüyor.
At least one dashboard and one alert rule visible.
- ☐ **Public URL veya çalışan demo**
/ping endpoint'inden pong dönüşünü gösteren video ya da görüntü.
A video or screenshot showing /ping returning pong.
- ☐ **RUNBOOK.md ve SECURITY.md**
Yeniden başlatma, geri alma, log bakma ve sır rotasyonu adımları.
Restart, rollback, looking at logs, and secret rotation steps.
- ☐ **Decisions log (ADR formatında)**
En az üç karar, her biri 3-5 cümle uzunluğunda.
At least three decisions, each 3–5 sentences long.

SUBMISSION FORMAT

Tek bir e-posta veya Slack mesajı ile gönderim yeterli: repo linki, kısa bir demo videosu ya da ekran görüntüsü ve varsa Public URL. Repo private ise `insider-one-devops` ekibini davet etmek pratik bir yöntem.

A single email or Slack message works fine: the repo link, a short demo video or screenshot, and the Public URL if available. If the repo is private, inviting the `insider-one-devops` team is a handy way to share it.

A few things to watch out for

Production sistemlerinde dikkat edilmesi gereken birkaç temel konu.

Aşağıdaki maddeler, gerçek bir production ortamında dikkat edilmesi gereken konular. Süreç boyunca bunlardan kaçınmak, hem güvenli bir teslim hem de iyi bir DevOps alışkanlığı için önemli.

The items below are things to be mindful of in a real production environment. Avoiding them throughout the process matters both for a secure submission and for healthy DevOps habits.

Watch out · Dikkat edilmesi gereken konular

- × Sırların açık metin olarak repo içinde tutulması
Keeping secrets as plaintext inside the repo
- × CI içine gizli değerlerin manuel olarak yapıştırılması
Pasting secret values manually into CI
- × Pipeline'daki imaj taraması adımının kapatılması veya kaldırılması
Turning off or removing the image scan step in the pipeline
- × Public repo üzerinden credential veya token sızıntısı
Leaking a credential or token through a public repo
- × Erişilemeyen ya da çalışmayan bir demo veya Public URL
A demo or Public URL that's unreachable or doesn't work
- × Çok geniş IAM tanımları (örneğin SSH'ın dünyaya açık olması)
Overly broad IAM definitions (for example, SSH open to the world)

Şüpheye düşersen sormak iyi bir yöntem. "Bunu yapsam riskli olur mu?" diye düşünmek zaten doğru bir refleks. Bir konuda emin değilsen README'de kısaca paylaşabilir veya bize sorabilirsiniz.

When in doubt, asking is a good move. Pausing to think "would this be risky?" is already a healthy instinct. If you're unsure about something, a short note in the README or a quick message to us works just as well.

Tools we'd reach for

Bir başlangıç noktası olarak düşünebilirsin; farklı bir araç tercih edersen nedenini README'de paylaşman yeterli.

TOOL	WHY	WHERE
minikube	Tek node'lu bir Kubernetes ortamı; laptop veya EC2 üzerinde dakikalar içinde ayağa kalkıyor.	minikube.sigs.k8s.io
Helm	Chart paketleme ve ortam ayrımı için yaygın bir seçim; production'da bir standart.	helm.sh/docs
GitHub Actions	Public repolarda dakika limiti oldukça geniş; OIDC desteği de mevcut.	docs.github.com/actions
Trivy	İmaj zafiyet taraması için pratik bir araç; CRITICAL ve HIGH bulgularda pipeline'ı kırmak kolay.	aquasecurity.github.io/trivy
gitleaks	Repo içinde sır taraması için pre-commit veya CI adımı olarak kullanılabilir.	github.com/gitleaks/gitleaks
kube-prometheus-stack	Prometheus, Grafana ve Alertmanager'ı tek Helm install ile getiriyor.	prometheus-community.github.io
ArgoCD / Flux	GitOps yaklaşımı için iki seçenek; ArgoCD bir UI sunarken Flux daha minimal kalıyor.	argo-cd.readthedocs.io · fluxcd.io
Terraform / OpenTofu	Track A için IaC seçeneği; AWS provider'ı free tier ile rahatlıkla çalışıyor.	developer.hashicorp.com · opentofu.org
ngrok / cloudflared	Track B için hızlı bir tunnel kurmanın iki yolu.	ngrok.com · developers.cloudflare.com
AWS EC2 free tier	Track A için EC2 erişim noktası; minikube'a uygun instance tipini free tier eligible olanlardan seçebilirsiniz.	aws.amazon.com/pm/ec2

WHEN IN DOUBT · ŞÜPHEYE DÜŞERSEN

Kullandığın bir araca yabancıysan kısa bir tutorial koşturup neyi neden tercih ettiğini bir ADR notuna dökmek pratik bir yol. Doğru cevap, açıklayabildiğin cevap.

If a tool is new to you, running a short tutorial and writing the choice and reasoning into an ADR note is a practical path. The right answer is the one you can explain.

BOL ŞANS · GOOD LUCK

Have fun with it.

Mükemmel olmasına gerek yok. Çalışan, tekrar üretilebilir ve neden öyle yaptığını anlatabildiğin küçük bir parça yeterli. Yolun keyifli geçsin.

It doesn't need to be perfect. A small, reproducible piece that works and that you can explain is plenty. Enjoy the journey along the way.

© 2026 Insider One. All rights reserved.

This document is confidential and intended solely for the recipient. Unauthorized use, disclosure, or distribution is prohibited.

INSIDER ONE · DevOps Team · insiderone.com